



VIT[®]
BHOPAL
www.vitbhopal.ac.in

Laboratory Record

CSA4029

Reinforcement and Representation Learning

Slot: B21+E14+E22

Interim Semester: 2025-26

Name of the student: Garv Anand

Register Number: 23BAI10605

Index

Exp. No.	Name of the Experiment	Page #
1.	Implement the Deep Deterministic Policy Gradient (DDPG) algorithm	3

Exp No: 4	Implement the Deep Deterministic Policy Gradient (DDPG) algorithm
Date:30/03/2026	

Aim:

To implement the Deep Deterministic Policy Gradient (DDPG) algorithm using PyTorch and train an agent on the continuous action-space environment Pendulum-v1, demonstrating convergence of cumulative episode rewards over 100 training episodes.

Objective / Algorithm:**1. Initialize Networks**

- Actor:

$$\pi(s | \theta^\pi)$$

- Critic:

$$Q(s, a | \theta^Q)$$

- Target Networks (same initial weights):

$$\theta^{\pi'} \leftarrow \theta^\pi, \theta^{Q'} \leftarrow \theta^Q$$

2. Initialize Replay Buffer & Noise

- Replay Buffer B , capacity = 50,000
- Ornstein-Uhlenbeck noise process $\mathcal{N}(t)$

3. For each episode (up to 100)

- Reset environment $\rightarrow$ initial state:

$$s_0$$

- Reset noise process

4. Action Selection

$$a_t = \pi(s_t | \theta^\pi) + \mathcal{N}(t)$$

- Clip a_t to valid action bounds

5. Environment Step

Execute a_t , observe:

$$(r_t, s_{t+1}, d_t)$$

6. Store Transition

$$(s_t, a_t, r_t, s_{t+1}, d_t) \in B$$

- If $|B| < 500$: skip training (warm-up)
-

7. Sample Mini-batch

Sample $N = 64$ transitions

Target Q-value:

$$y_i = r_i + \gamma \cdot Q'(s_{i+1}, \pi'(s_{i+1} | \theta^{\pi'}) | \theta^{Q'}) \cdot (1 - d_i)$$

where:

$$\gamma = 0.99$$

8. Critic Update (Loss Function)

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - Q(s_i, a_i | \theta^Q))^2$$

- Apply gradient clipping:

$$\|\nabla_{\theta^Q}\| \leq 1.0$$

9. Actor Update (Policy Gradient)

$$\nabla_{\theta^{\pi}} J \approx \frac{1}{N} \sum_{i=1}^N [\nabla_a Q(s, a | \theta^Q) |_{a=\pi(s_i)} \cdot \nabla_{\theta^{\pi}} \pi(s_i | \theta^{\pi})]$$

10. Soft Target Updates (Polyak Averaging)

$$\begin{aligned} \theta^{Q'} &\leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \\ \theta^{\pi'} &\leftarrow \tau \theta^{\pi} + (1 - \tau) \theta^{\pi'} \end{aligned}$$

where:

$$\tau = 0.005$$

11. Evaluation Metrics

- Episode reward:

$$R = \sum_t r_t$$

- Rolling average (10 episodes)
- Final metrics:
 - Average reward (last 20 episodes)
 - Best episode:

$$\max(R)$$

- Worst episode:

$$\min(R)$$

Program:

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import gymnasium as gym
import matplotlib.pyplot as plt
from collections import deque
import random

GAMMA = 0.99
TAU = 0.005
ACTOR_LR = 1e-4
CRITIC_LR = 1e-3
BUFFER_SIZE = 50000
BATCH_SIZE = 64
MAX_EPISODES = 100
MAX_STEPS = 200
```

```

WARMUP_STEPS = 500

HIDDEN = 256

SEED = 42

class ReplayBuffer:

    def __init__(self, max_size):

        self.buffer = deque(maxlen=max_size)

    def push(self, s, a, r, s_, d):

        self.buffer.append((s, a, r, s_, d))

    def sample(self, n):

        batch = random.sample(self.buffer, n)

        s, a, r, s_, d = zip(*batch)

        s = torch.FloatTensor(np.array(s))

        a = torch.FloatTensor(np.array(a))

        r = torch.FloatTensor(np.array(r)).unsqueeze(1)

        s_ = torch.FloatTensor(np.array(s_))

        d = torch.FloatTensor(np.array(d)).unsqueeze(1)

        return s, a, r, s_, d

    def __len__(self):

        return len(self.buffer)

class OUNoise:

    def __init__(self, dim):

        self.mu = np.zeros(dim)

        self.theta = 0.15

        self.sigma = 0.2

```

```

        self.state = np.copy(self.mu)

    def reset(self):

        self.state = np.copy(self.mu)

    def __call__(self):

        dx = self.theta * (self.mu - self.state) + self.sigma *
np.random.randn(len(self.mu))

        self.state += dx

        return self.state

class Actor(nn.Module):

    def __init__(self, s_dim, a_dim, a_high):

        super().__init__()

        self.a_high = torch.FloatTensor(a_high)

        self.fc1 = nn.Linear(s_dim, HIDDEN)

        self.fc2 = nn.Linear(HIDDEN, HIDDEN)

        self.fc3 = nn.Linear(HIDDEN, a_dim)

    def forward(self, x):

        x = F.relu(self.fc1(x))

        x = F.relu(self.fc2(x))

        x = torch.tanh(self.fc3(x)) * self.a_high

        return x

class Critic(nn.Module):

    def __init__(self, s_dim, a_dim):

        super().__init__()

        self.fc1 = nn.Linear(s_dim + a_dim, HIDDEN)

```

```

        self.fc2 = nn.Linear(HIDDEN, HIDDEN)

        self.fc3 = nn.Linear(HIDDEN, 1)

    def forward(self, s, a):

        x = torch.cat([s, a], dim=1)

        x = F.relu(self.fc1(x))

        x = F.relu(self.fc2(x))

        return self.fc3(x)

class DDPGAgent:

    def __init__(self, s_dim, a_dim, a_high, a_low):

        self.actor = Actor(s_dim, a_dim, a_high)

        self.critic = Critic(s_dim, a_dim)

        self.target_actor = Actor(s_dim, a_dim, a_high)

        self.target_critic = Critic(s_dim, a_dim)

        self.target_actor.load_state_dict(self.actor.state_dict())

        self.target_critic.load_state_dict(self.critic.state_dict())

        self.actor_optimizer = torch.optim.Adam(self.actor.parameters(),
lr=ACTOR_LR)

        self.critic_optimizer = torch.optim.Adam(self.critic.parameters(),
lr=CRITIC_LR)

        self.buffer = ReplayBuffer(BUFFER_SIZE)

        self.noise = OUNoise(a_dim)

        self.a_high = torch.FloatTensor(a_high)

        self.a_low = torch.FloatTensor(a_low)

    def _soft_update(self, net, target):

        for param, target_param in zip(net.parameters(), target.parameters()):

            target_param.data.copy_(TAU * param.data + (1 - TAU) *
target_param.data)

```

```

def act(self, state, explore=True):
    state_tensor = torch.FloatTensor(state).unsqueeze(0)
    with torch.no_grad():
        action = self.actor(state_tensor).squeeze(0).numpy()
    if explore:
        action += self.noise()
        action = np.clip(action, self.a_low.numpy(), self.a_high.numpy())
    return action

def remember(self, *args):
    self.buffer.push(*args)

def learn(self):
    if len(self.buffer) < WARMUP_STEPS:
        return
    s, a, r, s_, d = self.buffer.sample(BATCH_SIZE)
    with torch.no_grad():
        next_a = self.target_actor(s_)
        y = r + GAMMA * self.target_critic(s_, next_a) * (1 - d)
    critic_loss = F.mse_loss(self.critic(s, a), y)
    self.critic_optimizer.zero_grad()
    critic_loss.backward()
    torch.nn.utils.clip_grad_norm_(self.critic.parameters(), 1.0)
    self.critic_optimizer.step()
    actor_loss = -self.critic(s, self.actor(s)).mean()
    self.actor_optimizer.zero_grad()
    actor_loss.backward()
    self.actor_optimizer.step()

```

```

        self._soft_update(self.actor, self.target_actor)

        self._soft_update(self.critic, self.target_critic)

np.random.seed(SEED)

torch.manual_seed(SEED)

random.seed(SEED)

env = gym.make('Pendulum-v1')

s_dim = env.observation_space.shape[0]

a_dim = env.action_space.shape[0]

a_high = env.action_space.high

a_low = env.action_space.low

agent = DDPGAgent(s_dim, a_dim, a_high, a_low)

rewards = []

for episode in range(MAX_EPISODES):
    state, _ = env.reset(seed=SEED)

    agent.noise.reset()

    ep_reward = 0

    for step in range(MAX_STEPS):
        action = agent.act(state)

        next_state, reward, terminated, truncated, _ = env.step(action)

        done = terminated or truncated

        agent.remember(state, action, reward, next_state, float(done))

        agent.learn()

        state = next_state

        ep_reward += reward

        if done:

```

```

        break

    rewards.append(ep_reward)

    if (episode + 1) % 10 == 0:

        avg = np.mean(rewards[-10:])

        print(f'Episode {episode+1:>3} | Reward: {ep_reward:>8.2f} | Avg10:
{avg:>8.2f}')
```



```

rolling = [np.mean(rewards[max(0, i-9):i+1]) for i in range(len(rewards))]

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(rewards, color='#90CAF9', linewidth=0.8, alpha=0.5, label='Episode Reward')
ax.plot(rolling, color='#1565C0', linewidth=2.0, label='Rolling Avg (10)')

final_avg = np.mean(rewards[-20:])

ax.axhline(final_avg, color='#E53935', linestyle='--', linewidth=1.2, label=f'Final
Avg: {final_avg:.2f}')
```



```

ax.set_title('DDPG - Pendulum-v1', fontsize=14, fontweight='bold')

ax.set_xlabel('Episode')

ax.set_ylabel('Total Reward')

ax.legend()

ax.grid(True, alpha=0.3)

plt.tight_layout()

plt.savefig('ddpg_rewards.png', dpi=150)

plt.show()

print('=' * 50)

print(f'   Episodes      : {MAX_EPISODES}')
```



```

print(f'   Final Avg      : {np.mean(rewards[-20:]):.2f}')
```



```

print(f'   Best Episode   : {max(rewards):.2f}')
```



```

print(f'   Worst Episode: {min(rewards):.2f}')
```



```

print('=' * 50)
```

Output:

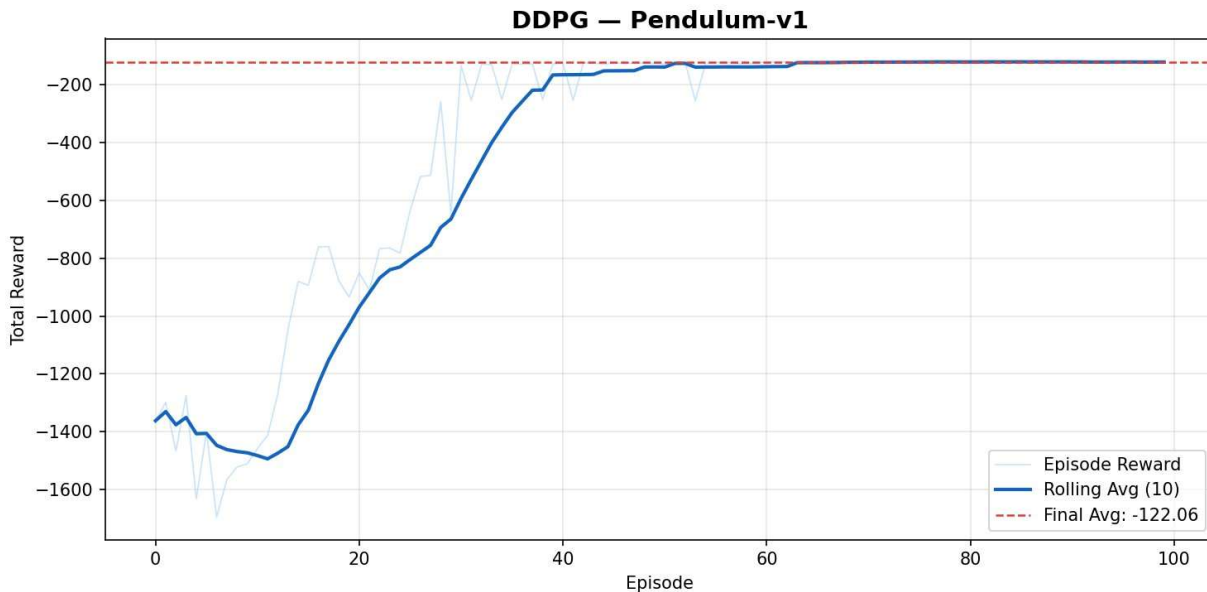


Fig 1: DDPG Training Curve on Pendulum-v1 (100 Episodes)

Result:

Thus the program is executed and output is verified.

The DDPG agent was successfully trained on Pendulum-v1. The rolling average reward improved from approximately **-1450** in the early episodes to **-122.06** (Final Average over last 20 episodes), demonstrating clear convergence of the policy. The agent learned to balance the pendulum through actor-critic updates with soft target network synchronisation.